```
# ================================
# INSTALL DEPENDENCIES
# ================================
!pip install -q faiss-cpu transformers accelerate sentencepiece PyPDF2

# ================================
# IMPORT LIBRARIES
# ================================
import faiss
import numpy as np
import torch
from transformers import AutoTokenizer, AutoModel,
AutoModelForCausalLM
from PyPDF2 import PdfReader
from google.colab import files

# ================================
# DEVICE
# ================================
device = "cuda" if torch.cuda.is_available() else "cpu"
print("Using device:", device)
```

```
Using device: cpu
```

```
# ================================
# LOAD SENTENCE EMBEDDING MODEL
# ================================
enc_name = "sentence-transformers/all-MiniLM-L6-v2"

enc_tokenizer = AutoTokenizer.from_pretrained(enc_name)
enc_model = AutoModel.from_pretrained(enc_name).to(device)

enc_model.eval()
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/
_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your
settings tab (https://huggingface.co/settings/tokens), set it as
secret in your Google Colab and restart your session.
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to
access public models or datasets.
  warnings.warn(
```


```
{"model_id":"9aa58e3b29234b28ae332381eaf63e3e","version_major":2,"vers
ion_minor":0}

{"model_id":"576bdfff3d934de9806b979f9e55af31","version_major":2,"vers
ion_minor":0}
```

```
Warning: You are sending unauthenticated requests to the HF Hub.
Please set a HF_TOKEN to enable higher rate limits and faster
downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending
unauthenticated requests to the HF Hub. Please set a HF_TOKEN to
enable higher rate limits and faster downloads.
```


```
{"model_id":"bf87485a910a4f4dac829691c7c5fdec","version_major":2,"vers
ion_minor":0}

{"model_id":"c4833db38235408895135df219d8e36d","version_major":2,"vers
ion_minor":0}

{"model_id":"b75232eeca064cd9b57430b89f78762a","version_major":2,"vers
ion_minor":0}

{"model_id":"2bb5bf41bb324b1b89908cea763e2a38","version_major":2,"vers
ion_minor":0}

{"model_id":"473a0e59e98b437d853f6e428c3cf4a0","version_major":2,"vers
ion_minor":0}
```


```
BertModel LOAD REPORT from: sentence-transformers/all-MiniLM-L6-v2
Key                     | Status     |  |
------------------------+------------+--+-
embeddings.position_ids | UNEXPECTED |  |

Notes:
- UNEXPECTED    :can be ignored when loading from different
task/architecture; not ok if you expect identical arch.

BertModel(
  (embeddings): BertEmbeddings(
    (word_embeddings): Embedding(30522, 384, padding_idx=0)
    (position_embeddings): Embedding(512, 384)
    (token_type_embeddings): Embedding(2, 384)
    (LayerNorm): LayerNorm((384,), eps=1e-12, elementwise_affine=True)
    (dropout): Dropout(p=0.1, inplace=False)
  )
  (encoder): BertEncoder(
    (layer): ModuleList(
      (0-5): 6 x BertLayer(
        (attention): BertAttention(
          (self): BertSelfAttention(
            (query): Linear(in_features=384, out_features=384,
bias=True)
            (key): Linear(in_features=384, out_features=384,
bias=True)
            (value): Linear(in_features=384, out_features=384,
bias=True)
            (dropout): Dropout(p=0.1, inplace=False)
```

```
          )
          (output): BertSelfOutput(
            (dense): Linear(in_features=384, out_features=384,
bias=True)
            (LayerNorm): LayerNorm((384,), eps=1e-12,
elementwise_affine=True)
            (dropout): Dropout(p=0.1, inplace=False)
          )
        )
        (intermediate): BertIntermediate(
          (dense): Linear(in_features=384, out_features=1536,
bias=True)
          (intermediate_act_fn): GELUActivation()
        )
        (output): BertOutput(
          (dense): Linear(in_features=1536, out_features=384,
bias=True)
          (LayerNorm): LayerNorm((384,), eps=1e-12,
elementwise_affine=True)
          (dropout): Dropout(p=0.1, inplace=False)
        )
      )
    )
  )
  (pooler): BertPooler(
    (dense): Linear(in_features=384, out_features=384, bias=True)
    (activation): Tanh()
  )
)

# ================================
# LOAD LLM (TinyLlama)
# ================================
llm_name = "TinyLlama/TinyLlama-1.1B-Chat-v1.0"

llm_tokenizer = AutoTokenizer.from_pretrained(llm_name)
llm_tokenizer.pad_token = llm_tokenizer.eos_token

llm_model = AutoModelForCausalLM.from_pretrained(
    llm_name,
    torch_dtype=torch.float16 if device == "cuda" else torch.float32,
    device_map="auto"
)

llm_model.eval()
```

{"model_id":"20593a73e05940a699f0a068112a9ccd","version_major":2,"version_minor":0}

{"model_id":"06c44d79ea044da9bf01ca4ff8ad7ddc","version_major":2,"version_minor":0}

{"model_id":"85a6c318246545abb0728e640debd5ba","version_major":2,"version_minor":0}

{"model_id":"7550aa2edca943dc9cf021f687242f50","version_major":2,"version_minor":0}

{"model_id":"ce6224f57efd4ff3a0b1eb69e195f8a1","version_major":2,"version_minor":0}

```
`torch_dtype` is deprecated! Use `dtype` instead!
```

{"model_id":"81066df7ac064cd9a6437c811f9929fb","version_major":2,"version_minor":0}

```
# ================================
# UPLOAD PDF
# ================================
uploaded = files.upload()
pdf_path = list(uploaded.keys())[0]
```

```
<IPython.core.display.HTML object>
```

```
Saving Exp 7.pdf to Exp 7.pdf
```

```
# ================================
# PDF TEXT EXTRACTION
# ================================
def extract_text_from_pdf(path):
    reader = PdfReader(path)
    text = ""

    for page in reader.pages:
        page_text = page.extract_text() or ""
        text += page_text + "\n"

    return text

# ================================
# TEXT CHUNKING
# ================================
def chunk_text(text, chunk_size=200, overlap=50):

    words = text.split()
    chunks = []

    start = 0

    while start < len(words):
```

```
        end = start + chunk_size
        chunk = " ".join(words[start:end])
        chunks.append(chunk)

        start += chunk_size - overlap

    return chunks

# ================================
# ENCODE CHUNKS
# ================================
def encode_chunks(chunks, batch_size=8):

    all_embeddings = []

    with torch.no_grad():

        for i in range(0, len(chunks), batch_size):

            batch = chunks[i:i+batch_size]

            tokens = enc_tokenizer(
                batch,
                padding=True,
                truncation=True,
                return_tensors="pt"
            ).to(device)

            outputs = enc_model(**tokens)

            embeddings = outputs.last_hidden_state.mean(dim=1)

            all_embeddings.append(
                embeddings.cpu().numpy().astype("float32")
            )

    return np.vstack(all_embeddings)

# ================================
# BUILD VECTOR DATABASE
# ================================
raw_text = extract_text_from_pdf(pdf_path)

doc_chunks = chunk_text(raw_text, chunk_size=200, overlap=50)

doc_embeddings = encode_chunks(doc_chunks)

faiss.normalize_L2(doc_embeddings)

dim = doc_embeddings.shape[1]
```

```
index = faiss.IndexFlatIP(dim)

index.add(doc_embeddings)

print("Total chunks indexed:", len(doc_chunks))
```

```
Total chunks indexed: 40
```

```
# ================================
# RETRIEVE CONTEXT
# ================================
def retrieve_context(query, top_k=5):

    with torch.no_grad():

        tokens = enc_tokenizer(
            [query],
            padding=True,
            truncation=True,
            return_tensors="pt"
        ).to(device)

        outputs = enc_model(**tokens)

        q_emb = outputs.last_hidden_state.mean(dim=1)

        q_emb = q_emb.cpu().numpy().astype("float32")

        faiss.normalize_L2(q_emb)

        scores, indices = index.search(q_emb, top_k)

    return [doc_chunks[i] for i in indices[0]]

# ================================
# RAG ANSWER GENERATION
# ================================
def generate_rag_answer(question, top_k=5, max_new_tokens=200):

    contexts = retrieve_context(question, top_k)

    context_text = "\n\n".join(contexts)

    prompt = f"""
You are an assistant that answers ONLY using the provided context.

Context:
{context_text}

Question: {question}
Answer:
```

```
"""

    inputs = llm_tokenizer(
        prompt,
        return_tensors="pt",
        truncation=True
    ).to(device)

    with torch.no_grad():

        output_ids = llm_model.generate(
            **inputs,
            max_new_tokens=max_new_tokens,
            do_sample=False
        )

    output_text = llm_tokenizer.decode(
        output_ids[0],
        skip_special_tokens=True
    )

    return output_text.split("Answer:")[-1].strip()

# ================================
# RUN QUERY
# ================================
question = "What is the main contribution of this paper?"

answer = generate_rag_answer(question)

print("\nQuestion:", question)
print("\nAnswer:", answer)
```

```
This is a friendly reminder - the current text generation call has
exceeded the model's predefined maximum length (2048). Depending on
the model, you may observe exceptions, performance degradation, or
nothing at all.


Question: What is the main contribution of this paper?

Answer: You are an assistant that answers ONLY using the provided
context.

Context:
and Beatrice Santorini. Building a large annotated corpus of english:
The penn treebank. Computational linguistics , 19(2):313–330, 1993.
[26] David McClosky, Eugene Charniak, and Mark Johnson. Effective
self-training for parsing. In Proceedings of the Human Language
Technology Conference of the NAACL, Main Conference , pages 152–159.
ACL, June 2006. [27] Ankur Parikh, Oscar Täckström, Dipanjan Das, and
```

Jakob Uszkoreit. A decomposable attention model. In Empirical Methods in Natural Language Processing , 2016. [28] Romain Paulus, Caiming Xiong, and Richard Socher. A deep reinforced model for abstractive summarization. arXiv preprint arXiv:1705.04304 , 2017. [29] Slav Petrov, Leon Barrett, Romain Thibaux, and Dan Klein. Learning accurate, compact, and interpretable tree annotation. In Proceedings of the 21st International Conference on Computational Linguistics and 44th Annual Meeting of the ACL , pages 433–440. ACL, July 2006. [30] Ofir Press and Lior Wolf. Using the output embedding to improve language models. arXiv preprint arXiv:1608.05859 , 2016. [31] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. arXiv preprint arXiv:1508.07909 , 2015. [32] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. arXiv preprint arXiv:1701.06538

pages 434–443. ACL, August 2013. 12 Attention Visualizations Input-Input Layer5 It is in this spirit that a majority of American governments have passed new laws since 2009 making the registration or voting process more difficult . <EOS> <pad> <pad> <pad> <pad> <pad> <pad> It is in this spirit that a majority of American governments have passed new laws since 2009 making the registration or voting process more difficult . <EOS> <pad> <pad> <pad> <pad> <pad> <pad> Figure 3: An example of the attention mechanism following long-distance dependencies in the encoder self-attention in layer 5 of 6. Many of the attention heads attend to a distant dependency of the verb ‘making’, completing the phrase ‘making...more difficult’. Attentions here shown only for the word ‘making’. Different colors represent different heads. Best viewed in color. 13 Input-Input Layer5 The Law will never be perfect , but its application should be just - this is what we are missing , in my opinion . <EOS> <pad> The Law will never be perfect , but its application should be just - this is what we are missing , in my opinion . <EOS> <pad> Input-Input Layer5 The Law will never be perfect , but its

Recognition , pages 770–778, 2016. [12] Sepp Hochreiter, Yoshua Bengio, Paolo Frasconi, and Jürgen Schmidhuber. Gradient flow in recurrent nets: the difficulty of learning long-term dependencies, 2001. [13] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. Neural computation , 9(8):1735–1780, 1997. [14] Zhongqiang Huang and Mary Harper. Self-training PCFG grammars with latent annotations across languages. In Proceedings of the 2009 Conference on Empirical Methods in Natural Language Processing , pages 832–841. ACL, August 2009. [15] Rafal Jozefowicz, Oriol Vinyals, Mike Schuster, Noam Shazeer, and Yonghui Wu. Exploring the limits of language modeling. arXiv preprint arXiv:1602.02410 , 2016. [16] Łukasz Kaiser and Samy Bengio. Can active memory replace attention? In Advances in Neural Information Processing Systems, (NIPS) , 2016. [17] Łukasz Kaiser and

Ilya Sutskever. Neural GPUs learn algorithms. In International Conference on Learning Representations (ICLR) , 2016. [18] Nal Kalchbrenner, Lasse Espeholt, Karen Simonyan, Aaron van den Oord, Alex Graves, and Ko- ray Kavukcuoglu. Neural machine translation in linear time. arXiv preprint arXiv:1610.10099v2 , 2017. [19] Yoon Kim, Carl Denton, Luong Hoang, and Alexander M. Rush. Structured attention networks. InInternational Conference on Learning Representations , 2017. [20] Diederik Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In ICLR

pages 3104–3112, 2014. [36] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jonathon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision. CoRR , abs/1512.00567, 2015. [37] Vinyals & Kaiser, Koo, Petrov, Sutskever, and Hinton. Grammar as a foreign language. In Advances in Neural Information Processing Systems , 2015. [38] Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc V Le, Mohammad Norouzi, Wolfgang Macherey, Maxim Krikun, Yuan Cao, Qin Gao, Klaus Macherey, et al. Google’s neural machine translation system: Bridging the gap between human and machine translation. arXiv preprint arXiv:1609.08144 , 2016. [39] Jie Zhou, Ying Cao, Xuguang Wang, Peng Li, and Wei Xu. Deep recurrent models with fast-forward connections for neural machine translation. CoRR , abs/1606.04199, 2016. [40] Muhua Zhu, Yue Zhang, Wenliang Chen, Min Zhang, and Jingbo Zhu. Fast and accurate shift-reduce constituent parsing. In Proceedings of the 51st Annual Meeting of the ACL (Volume 1: Long Papers) , pages 434–443. ACL, August 2013. 12 Attention Visualizations Input-Input Layer5 It is in this spirit that a majority of American governments have passed new laws since 2009 making the registration or voting process more difficult . <EOS> <pad> <pad> <pad> <pad> <pad> <pad> It is in this spirit that a

and Ko- ray Kavukcuoglu. Neural machine translation in linear time. arXiv preprint arXiv:1610.10099v2 , 2017. [19] Yoon Kim, Carl Denton, Luong Hoang, and Alexander M. Rush. Structured attention networks. InInternational Conference on Learning Representations , 2017. [20] Diederik Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In ICLR , 2015. [21] Oleksii Kuchaiev and Boris Ginsburg. Factorization tricks for LSTM networks. arXiv preprint arXiv:1703.10722 , 2017. [22] Zhouhan Lin, Minwei Feng, Cicero Nogueira dos Santos, Mo Yu, Bing Xiang, Bowen Zhou, and Yoshua Bengio. A structured self-attentive sentence embedding. In Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing , pages 111–119. ACL, July 2017. [23] Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc V Le, Mohammad Norouzi, Wolfgang Macherey, Maxim Krikun, Yuan Cao, Qin Gao, Klaus Macherey, et al. Google’s neural machine translation system: Bridging the gap between human and machine translation. arXiv preprint arXiv:1609.08144 , 2016. [24] Yonghui Wu, Mike Schuster, Zhifeng Xiong, and Yonghui Wu. A deep learning. In Proceedings of the 2017. [25] Yonghui Wu, Mike